

Refactoring del sistema de filtros

Índice

1. Motivación
2. Resumen de cambios
3. Trait PHP — HasHaversine
4. Partial Twig — sidebar de filtros
5. Partial Twig — script de proximidad
6. Qué NO se cambió y por qué
7. Cómo extender el sistema a una nueva entidad

1. Motivación

Al implementar el sistema de filtros en las vistas de **músicos** y **bandas** por separado, surgieron tres duplicaciones exactas:

Duplicación	Dónde	Líneas afectadas
<code>haversine()</code> + <code>calcularDistanciaKm()</code>	<code>MusicoRepository</code> y <code>BandaRepository</code>	~10 líneas × 2
HTML del sidebar de filtros	<code>musico/index.html.twig</code> y <code>banda/index.html.twig</code>	~70 líneas × 2
Bloque <code><script></code> (slider + geolocalización)	<code>musico/index.html.twig</code> y <code>banda/index.html.twig</code>	~75 líneas × 2

El refactoring **no altera ninguna funcionalidad ni apariencia visual**. Únicamente mueve código ya existente a ubicaciones compartidas.

2. Resumen de cambios

Archivos nuevos

Archivo	Tipo	Propósito
<code>src/Repository/Traits/HasHaversine.php</code>	PHP Trait	Fórmula de Haversine compartida entre repositorios
<code>templates/_filtros_sidebar.html.twig</code>	Partial Twig	Sidebar de filtros parametrizable
<code>templates/_filtros_proximidad_script.html.twig</code>	Partial Twig	Script JS del slider y geolocalización

Archivos modificados

Archivo	Cambio
src/Repository/MusicoRepository.php	use HasHaversine + eliminados métodos duplicados
src/Repository/BandaRepository.php	use HasHaversine + eliminados métodos duplicados
templates/musico/index.html.twig	Sidebar y script sustituidos por {% include %}
templates/banda/index.html.twig	Sidebar y script sustituidos por {% include %}

3. Trait PHP — HasHaversine

Archivo: src/Repository/Traits/HasHaversine.php

Qué es un trait en PHP

Un trait es un mecanismo de **reutilización de código horizontal** en PHP. Permite definir métodos una sola vez y "mezclarlos" en varias clases sin necesidad de herencia. La clase que lo usa escribe `use NombreDelTrait;` dentro de su cuerpo.

```
namespace App\Repository\Traits;

trait HasHaversine
{
    public function calcularDistanciaKm(float $lat1, float $lng1, float $lat2,
float $lng2): float
    {
        return $this->haversine($lat1, $lng1, $lat2, $lng2);
    }

    private function haversine(float $lat1, float $lng1, float $lat2, float
$lng2): float
    {
        $dLat = deg2rad($lat2 - $lat1);
        $dLng = deg2rad($lng2 - $lng1);
        $a = sin($dLat / 2) ** 2 + cos(deg2rad($lat1)) * cos(deg2rad($lat2)) *
sin($dLng / 2) ** 2;
        return 6371 * 2 * atan2(sqrt($a), sqrt(1 - $a));
    }
}
```

Uso en los repositorios

```
// MusicoRepository.php y BandaRepository.php (idéntico en ambos)
use App\Repository\Traits\HasHaversine;

class MusicoRepository extends ServiceEntityRepository
{
    // ...
```

```
use HasHaversine; // ← importa calcularDistanciaKm() y haversine()
...
}
```

Con esto, `$musicoRepository->calcularDistanciaKm(...)` y `$bandaRepository->calcularDistanciaKm(...)` siguen funcionando exactamente igual que antes desde los controladores. El contrato público no cambia.

Por qué un trait y no una clase base o un servicio

Opción	Ventaja	Inconveniente
Trait	Mínimo boilerplate; se mezcla directamente; no rompe la jerarquía de herencia	Solo adecuado para métodos sin dependencias externas
Clase base intermedia	Centraliza herencia	Rompe la cadena <code>ServiceEntityRepository</code> → <code>clase base</code> → <code>repositorio</code> ; Doctrine no lo espera
Servicio <code>GeoService</code>	Más testeable de forma aislada	Requiere inyección de dependencias en los repositorios y en los controladores

El trait es la opción con menor impacto estructural para una función pura sin estado (solo matemáticas).

4. Partial Twig — sidebar de filtros

Archivo: `templates/_filtros_sidebar.html.twig`

El prefijo `_` en el nombre del archivo es una **convención** para indicar que es un partial (fragmento incluido por otras plantillas) y no una vista completa renderizable directamente.

Variables que acepta

Variable	Tipo	Obligatoria	Descripción
<code>route</code>	<code>string</code>	Sí	Nombre de la ruta Symfony para el <code>action</code> del form y el enlace "Limpiar"
<code>generos</code>	<code>Genero[]</code>	Sí	Colección de géneros para los checkboxes
<code>filtros</code>	<code>array</code>	Sí	Valores actuales de los filtros (para repopular el form)
<code>instrumentos</code>	<code>InstrumentoSistema[]</code>	No	Si se omite o está vacía, el bloque de instrumentos no se renderiza

Lógica condicional para instrumentos

```
{% set instrumentos = instrumentos ?? [] %}

{% if instrumentos|length > 0 %}
    {# bloque de instrumentos #}
{% endif %}
```

`instrumentos ?? []` convierte una variable no definida en array vacío, evitando errores con `only`. El bloque HTML solo existe en el DOM renderizado cuando se pasa la variable con contenido — en el caso de bandas, simplemente no aparece.

Uso en las plantillas

```
{# musico/index.html.twig – con instrumentos #}
{% include '_filtros_sidebar.html.twig' with {
    route: 'app_musico_index',
    generos: generos,
    filtros: filtros,
    instrumentos: instrumentos
} only %}

{# banda/index.html.twig – sin instrumentos #}
{% include '_filtros_sidebar.html.twig' with {
    route: 'app_banda_index',
    generos: generos,
    filtros: filtros
} only %}
```

La palabra clave `only`

`only` restringe el contexto del partial a **únicamente** las variables pasadas en `with`. Las variables del contexto padre (como `bandas`, `musicos`, `distancias`) no son accesibles dentro del partial, lo que:

- Hace explícitas las dependencias del partial.
- Previene que el partial use accidentalmente variables del padre.
- Las **variables globales de Twig** (como `google_places_api_key`, registrada en `twig.yaml`) siguen siendo accesibles aunque se use `only`.

5. Partial Twig — script de proximidad

Archivo: `templates/_filtros_proximidad_script.html.twig`

Contiene el bloque `<script>` completo con el IIFE que gestiona:

1. El degradado visual del slider de radio.
2. La sincronización del estado habilitado/deshabilitado del slider con el campo de ciudad.
3. El botón "Usar mi ubicación" (Geolocation API + reverse geocoding).

Por qué es un partial Twig y no un archivo .js

El script **no contiene variables Twig** (no usa `{{ }}` ni `{% %}`), por lo que podría vivir en un `.js` estático. Sin embargo, la convención de este proyecto es incluir scripts de página inline al final del `{% block body %}`, y un partial Twig mantiene esa coherencia sin introducir un nuevo sistema de carga de assets (Webpack Encore, importmap, etc.).

Si en el futuro se adopta un bundler, mover el contenido a un `.js` es trivial.

Uso en las plantillas

```
{# Al final de {% block body %}, antes de {% endblock %} #}  
{% include '_filtros_proximidad_script.html.twig' %}
```

No se usa `only` aquí porque el partial no necesita ninguna variable Twig — es HTML puro con JavaScript. Con o sin `only` el resultado es idéntico.

6. Qué NO se cambió y por qué

Elemento	Razón para no tocar
Lógica de <code>findByFiltros()</code> en cada repositorio	Cada entidad tiene sus propias joins y campos (<code>instrumentosSistema</code> en Musico vs. nada en Banda); no es código idéntico
Parseo de parámetros en los controladores	Aunque similar, difieren en <code>\$instrumentoIds</code> (Musico sí, Banda no); extraerlo añadiría complejidad sin beneficio real
CSS (<code>perfil.css</code>)	Ya era compartido; no hay duplicación que resolver
Controlador Stimulus	Ya era único y compartido por ambas vistas
Distancia en tarjeta (Twig)	Es una sola línea por plantilla; no justifica un partial

7. Cómo extender el sistema a una nueva entidad

Si en el futuro se añade, por ejemplo, una vista de `Sala` o `Evento` con filtros de proximidad:

Backend

```
// src/Repository/SalaRepository.php  
use App\Repository\Traits\HasHaversine;  
  
class SalaRepository extends ServiceEntityRepository  
{  
    use HasHaversine;  
  
    public function findByFiltros(array $generoIds, ?float $lat, ?float $lng, ?int
```

```
$radio): array
{
    // query builder + filtro haversine
}
}
```

`calcularDistanciaKm()` estará disponible automáticamente sin escribir ni una línea extra.

Frontend

```
{# sala/index.html.twig #}
{% include '_filtros_sidebar.html.twig' with {
    route: 'app_sala_index',
    generos: generos,
    filtros: filtros
} only %}

{# ... grid de tarjetas ... #}

{% include '_filtros_proximidad_script.html.twig' %}
```

El sidebar y el script funcionan desde el primer momento. Solo hay que asegurarse de que el controlador pasa `generos`, `filtros` y `distancias` a la plantilla.